📘 ESP32 Air Mouse Project Manual

A Complete Technical Guide for Design, Implementation, and Testing

---

1. INTRODUCTION

1.1 Background

Traditional input devices like the mouse require a flat surface and physical interaction. With advancements in embedded systems and sensor technologies, it is now possible to design touchless human-computer interfaces.

The ESP32 Air Mouse project is a wearable system that allows users to control a computer cursor using hand gestures. It combines motion sensing, signal processing, and wireless communication into a compact embedded system.

1.2 Objectives

The primary goals of this project are:

- Design a gesture-based mouse system
- Implement sensor fusion using Kalman filtering
- Enable Bluetooth communication (BLE HID)
- Develop a portable and wearable device
- Ensure smooth and accurate cursor control

1.3 Applications

- Assistive technology for people with disabilities
- Smart home control systems
- Gaming and virtual reality interfaces
- Presentation control systems
- Human-computer interaction research

2. 🧰 SYSTEM OVERVIEW

2.1 System Components

2.1.1 ESP32 Microcontroller

- Dual-core processor
- Built-in Wi-Fi and Bluetooth
- Handles:
    - Sensor data processing
    - BLE communication
    - System control

2.1.2 MPU6050 Sensor

- 3-axis accelerometer
- 3-axis gyroscope
- Measures:
    - Tilt (orientation)
    - Rotation

2.1.3 LDR (Light Dependent Resistor)

- Detects light intensity
- Used for:
    - Gesture-based clicking

2.1.4 Trimpot (Replaces Fixed Resistor)

- Adjustable resistance
- Allows fine-tuning of click sensitivity

2.1.5 Push Button

- Used to:
    - Toggle system ON/OFF

2.1.6 Breadboard / Perfboard

- Breadboard → Testing
- Perfboard → Final compact build

3. HARDWARE DESIGN

3.1 Power Requirements

- Operating Voltage: 3.3V
- Power Source:
    - USB (development)
    - Battery (portable)

3.2 Circuit Design

3.2.1 MPU6050 (I2C Interface)

Signal	ESP32 Pin
SDA	GPIO 21
SCL	GPIO 22

3.2.2 LDR Circuit (Voltage Divider)

```
3.3V ———— LDR ———— GPIO 34
          |
        Trimpot
          |
         GND
```

3.2.3 Button Circuit

GPIO 14 — Button — GND

- Uses internal pull-up resistor

3.3 Wearable Design Considerations

- Place MPU6050 under ESP32
- Minimize wiring length
- Keep LDR exposed
- Use compact layout
- Ensure comfort under palm

4. SOFTWARE DESIGN

4.1 Programming Environment

- Language: MicroPython
- IDE: Thonny / VS Code
- Libraries:
    - MPU6050 driver
    - Kalman filter
    - BLE HID (Mouse)

4.2 System Flow

Initialize → Calibrate → Connect Bluetooth → Read Sensors → Process Data → Send Mouse Commands

4.3 Code Structure

Main Modules:

- main.py → Core logic
- MPU6050.py → Sensor driver
- Kalman.py → Filtering
- hid_services.py → BLE mouse

5. ALGORITHM DESIGN

5.1 Sensor Fusion

Accelerometer:

- Measures tilt angle
- Noisy but stable

Gyroscope:

- Measures rotation speed
- Smooth but drifts

5.2 Kalman Filter

Combines both sensors to produce:

- Smooth
- Accurate
- Drift-free angles

5.3 Angle Computation

Roll = atan2(-Ay, -Az)

Pitch = atan2(-Ax, -Az)

5.4 Cursor Mapping

dx = roll - center_roll

dy = pitch - center_pitch

5.5 Movement Processing

- Apply dead zone
- Scale with sensitivity
- Clamp values

5.6 Click Detection

```
if LDR < threshold:
  trigger click
```

6. Bluetooth Communication

6.1 BLE HID Overview

- Device behaves as:
  → Bluetooth Mouse

6.2 Data Sent

- X movement
- Y movement
- Button clicks

6.3 Connection Process

1. Start advertising
2. Pair with computer
3. Send HID reports

7. Implementation Steps

Step 1: Assemble Hardware

- Connect MPU6050
- Build LDR circuit
- Add button

Step 2: Flash MicroPython

- Install firmware
- Verify board connection

Step 3: Upload Files

- Copy all required .py files

Step 4: Run System

- Execute main.py
- Observe serial output

Step 5: Pair Device

- Connect via Bluetooth
- Select “ESP32 Air Mouse”

8. Testing and Validation

8.1 Unit Testing

MPU6050 Test

- Verify angle changes

LDR Test

- Cover sensor → detect change

Button Test

- Press → toggle system

8.2 System Testing

- Cursor moves smoothly
- Click works reliably
- Bluetooth remains stable

9. Challenges and Solutions

9.1 Noise in Sensor Data

✓ Solution: Kalman filtering

9.2 Cursor Drift

✓ Solution: Calibration + offsets

9.3 False Clicks

✓ Solution: Adjust threshold using trimpot

9.4 Bluetooth Instability

✓ Solution:

- Restart ESP32
- Ensure strong signal

10. Future Improvements

- Right-click gesture
- Scroll gesture
- Gesture recognition (AI-based)
- Rechargeable battery system
- Custom PCB design
- 3D printed casing

11. Conclusion

The ESP32 Air Mouse project successfully demonstrates:

- Integration of sensors and microcontrollers
- Real-time signal processing
- Wireless communication using BLE

- Development of a wearable interface

This project bridges the gap between hardware design and intelligent interaction systems, making it a strong foundation for advanced embedded and IoT applications.